导读

> 这本书关心的不是“模型会不会写代码”，而是“一个会写代码的模型被放进终端、仓库和团队流程以后，怎样才不会把系统带偏”。

这不是源码注释汇编，也不是产品功能介绍。它关注的是 Claude Code 如何把不稳定模型收束进可持续运行的工程秩序，让控制面、主循环、工具权限、上下文治理、恢复路径、多代理验证与团队制度长成一套完整骨架。

本书有三个阅读前提：

- 重点不在模型能力，而在 harness 如何组织约束与执行
- 重点不在函数逐条解释，而在运行时结构为什么必须这样长出来
- 重点不在个人技巧，而在这些结构怎样变成团队可以复用的制度

建议阅读顺序：

1. 序言 Harness、终端与工程约束
2. 第 1 章为什么需要 Harness Engineering
3. 第 2 章 Prompt 不是人格，Prompt 是控制平面
4. 第 3 章 Query Loop：代理系统的心跳
5. 第 4 章工具、权限与中断：为什么代理不能直接碰世界
6. 第 5 章上下文治理：Memory、CLAUDE.md 与 Compact 是预算制度
7. 第 6 章错误与恢复：出错后仍能继续工作的代理系统
8. 第 7 章多代理与验证：用分工和验证管理不稳定性
9. 第 8 章团队落地：把一个聪明工具变成可复用制度
10. 第 9 章 Harness Engineering 十条原则
11. 附录 A 检查清单：把原则落成能执行的约束
12. 附录 B 图示：把运行时骨架画出来
13. 附录 C 源码地图：本书各章主要依据哪些文件

如果只想先看总判断，可以直接跳到第 9 章。

序言 Harness、终端与工程约束

这些年，人们喜欢把会写代码的模型叫作智能体。这个词带着明显的乐观色彩，仿佛只要模型能读仓库、调工具、写出像样的 patch，它就可以在工程环境里独立行动。可工程环境有明确后果。终端、文件系统和 Git 历史都不是抽象空间，任何改动都会留下痕迹。

一个只会输出文本的模型，出错时主要带来理解成本。一个能运行命令、写文件、访问网络、修改仓库的模型，出错后留下的是执行结果。目录会变化，进程会中断，配置会损坏，历史会变得难以追踪。到了这一步，核心问题不再是模型是否足够聪明，而是系统是否提供了足够约束。

这本书讨论的，正是这种约束。

我把它叫作 Harness Engineering。这里的 harness 可以理解为一组持续生效的控制结构，用来约束模型在工程环境中的行为边界。对 AI coding agent 来说，没有约束的能力只会扩大事故半径。

这本书也不是一份 Claude Code 源码讲解。源码当然重要，但如果只沿着目录逐个解释，很容易写成注释汇编。那样的内容能说明函数做了什么，却未必能回答系统为什么必须长成现在这样。要理解 Claude Code 这类系统，光知道有 `queryLoop()`、`compactConversation()`、`runTools()` 还不够。更重要的问题是，为什么一个“会写代码”的系统最终需要 prompt 分层、权限判定、状态机、compact、恢复分支、subagent 生命周期、verification 阶段和团队制度这些结构。

这里也先说明一个边界：这份内容不会附带 Claude Code 源代码，也不会长篇逐段转录源码。原因很简单，就是版权边界。我们能做的，是在合理引用和工程分析的范围内，基于源码结构提炼设计原则、运行机制与方法论判断，而不是把受版权保护的实现文本重新发布一遍。

答案并不复杂，因为模型本身并不稳定。

这个判断未必讨喜，但工程系统不能依赖乐观叙事维持。一个部件如果本质上不稳定，系统就必须围绕这个事实设计；否则，问题只会在事故复盘里集中出现。

Claude Code 值得研究，因为它在实现上保持了明确的工程克制。它没有假定模型会持

续正确，因此用 query loop 管理状态；没有假定工具调用天然安全，因此用权限和调度约束工具；没有假定上下文越多越好，因此引入 memory、CLAUDE.md、compact 和 session memory；没有把错误视为偶发事件，因此为 prompt too long、max output tokens、中断和 hook 回环设计恢复路径；也没有把多代理直接等同于更强能力，因此把 synthesis 和 verification 单独拆开，避免系统自我背书。

这一整套东西，合起来才是 agent。模型只是 agent 里最会说话、也最不稳定的那个部件。

所以这本书有一个始终不变的基本立场：

> Prompt 决定它怎么说话，Harness 决定它怎么做事。

这里说的 harness，不是一层附属工具，也不是对模型能力的情绪化防御。它是模型进入工程环境的前提。缺少这层约束，风险最终会转移给用户、团队和未来的维护者。

本书试图做两件事。

第一件，是基于 Claude Code 源码，把真正决定系统可靠性的结构讲清楚。重点在于解释为什么上下文治理必须成为主路径，为什么多代理解决的是职责分区，为什么团队制度必须挂到生命周期节点上，而不是简单罗列“这里有 compact”“这里有 subagent”“这里有 hook”。

第二件，是把这些实现背后的判断提炼成更一般的工程原则。具体代码版本会变化，函数名会变化，产品形态也会变化。但只要大家还在尝试把不稳定模型接入真实工作流，某些原则就仍然有效。比如：

- 错误路径要按主路径设计
- 验证必须进入完成定义
- 权限是系统器官，而不是附属功能
- 上下文是资源，不是垃圾桶
- 多代理要靠角色分离，不靠人海战术
- 团队制度比个人技巧重要

如果这些判断成立，那么 Claude Code 更适合作为一份样本。它的价值不在于教人复制一套一模一样的 CLI，而在于展示一个面向真实工程环境的 AI 代理，最终会如何走向更严格的约束结构。

更直接地说，这本书不讨论如何把模型包装成一个“像工程师”的幻觉，而讨论如何在模型并不具备工程师稳定性的前提下，仍然构造出一个可运行的工程系统。

这类工作通常不显眼。回滚、审批、权限、验证、compact、清理孤儿进程都不显眼，但系统能否长期稳定运行，往往取决于这些部分。过度追求“像人一样自然”的代理体验，常见结果是系统具备了类似人的失误模式，却没有承担后果的能力。

既然如此，就从约束谈起。

接下来九章，讨论的都是这套 Harness 结构如何长出来，为什么必须这样长，以及一个团队如何把个人经验沉淀成可以复用的工程制度。

@wquguru
2026.04.01
Claude Code 源码泄漏的愚人节

btw. 您可以在 harness-books.agentway.dev/book1-claude-code 访问在线版，获取更好的阅读体验

第 1 章为什么需要 Harness Engineering

1.1 问题在于让模型别乱来

这些年，人们很喜欢谈智能体。这个词常常带着轻快的预期，仿佛只要模型会写几段代码、会调几个工具，就可以像见习工程师一样在终端里独立工作。可终端和文件系统都带有明确后果。一个会说话的概率分布，一旦能接触 shell、Git、网络和本地文件，问题就从“回答得不够好”变成“执行把东西改坏了”。

所以问题的重点，一直是怎样把它约束成一个可管理的系统。所谓 Harness Engineering，讨论的就是这件事。Harness 是一整套制度化的控制平面，用来处理一个很现实的问题：模型并不天然值得信任。

这个判断未必轻松，但通常有用。一个代理系统要进入真实工程环境，首先要承认自己的核心部件是不稳定的。忽视这一点，问题最后通常会在日志和事故记录里出现。

1.2 Claude Code 的第一层 Harness：受约束的会话系统

如果只看表面，Claude Code 像一个能和用户对话、还能顺手改代码的 CLI。可从实现上看，它一开始就没有把自己当成“裸模型接口”来设计，而是当成一个带有上下文边界、运行时状态和行为规约的会话系统。

这一点从 system prompt 的组织方式就能看出来。

在 `src/constants/prompts.ts:175` 开始，系统先定义身份和总任务；在 `src/constants/prompts.ts:186` 开始，补上关于工具、权限、系统提醒和上下文压缩的系统级说明；在 `src/constants/prompts.ts:199` 开始，再补上做任务时的工程约束，比如不要越权改动、不要把验证说成已经完成、不要为了省事发明抽象。

这里值得停一下。很多人谈 prompt，还停留在“你是一个什么样的助手”这种修辞层

面。Claude Code 在实现上把 prompt 放进了运行时控制结构里，这些文字用来规定执行边界、失败行为和报告责任。

更重要的是，这个 prompt 采用分段拼装方式。在 `src/constants/prompts.ts:444` 的 `getSystemPrompt()` 里，静态部分和动态部分被明确拆开；memory、language、output style、MCP instructions、scratchpad 等内容按段注入。到了 `src/utils/systemPrompt.ts:28`，系统又把默认 prompt、自定义 prompt、agent prompt 和 append prompt 组织成一套优先级规则。

这说明了一个朴素的工程事实：一个真正可用的代理系统，不能依赖一段“万能提示词”解决所有问题。它必须把控制拆成层，把层次拆成职责。否则，新增提醒和禁令很快就会互相冲突，系统行为也会变得难以预测。

1.3 第二层 Harness：代理依赖持续循环

如果说 prompt 规定了它应该成为什么样的东西，那么 query loop 规定了它实际上如何运行。

Claude Code 的核心不在某个单独的 API 调用，而在 `src/query.ts:219` 开始的 `query()`，以及 `src/query.ts:241` 开始的 `queryLoop()`。这段实现里最重要的一点，是它明确承认代理系统依赖带状态的多轮执行。

在 `src/query.ts:268`，系统把 `messages`、`toolUseContext`、`autoCompactTracking`、`maxOutputTokensRecoveryCount`、`hasAttemptedReactiveCompact`、`pendingToolUseSummary`、`turnCount`、`transition` 等内容放进同一个跨迭代状态里。一个会话系统一旦这样设计，就等于正式承认上一轮留下的问题会进入下一轮，需要继续处理。

这是 Harness 思维的核心。真正的问题在于系统能不能在连续多轮里保持行为一致：有没有预算概念，有没有恢复概念，有没有上下文膨胀后的自救机制，有没有在工具调用失败后继续推进任务的能力。缺少这些结构，所谓智能体就只是一个不稳定的执行者。

在 `src/query.ts:365` 往后，这个循环还会在每轮调用前处理消息裁剪、tool result budget、history snip、microcompact、context collapse、autocompact 等内容。实现细节虽然很多，但共同指向一点：Claude Code 在调用发生前就尽量把控制权收回到运行时一侧。

这也是为什么 Harness Engineering 不能被看作 prompt engineering 的附属品。前者关心状态机，后者关心措辞。措辞当然重要，但状态机决定系统行为最终由谁负责。

1.4 第三层 Harness：工具调用必须服从调度

一个模型如果只能输出文本，顶多让人觉得它有时说得太满。可一旦它能调用工具，系统风险就立刻从修辞风险变成执行风险。这时候最重要的问题是：谁决定工具怎么跑。

Claude Code 给出的答案很直接。运行时会根据工具属性决定并发还是串行。

在 `src/services/tools/toolOrchestration.ts:19` 的 `runTools()` 里,工具调用先经过 `partitionToolCalls()` 分组。到了 `src/services/tools/toolOrchestration.ts:91` 系统会读取工具 schema, 并调用 `isConcurrencySafe()` 判断一个工具是否适合并发执行。能并发的归成一批,不能并发的按顺序一个个来。并发路径里,context modifier 会先缓存,再按原始 block 顺序回放,见 `src/services/tools/toolOrchestration.ts:31` 到 `:63`。

这件事很有代表性。它说明 Claude Code 没有把工具当成模型能力的自然延伸，而是当成需要调度纪律的受管执行单元。缺少调度纪律的工具系统，只会把模型的不稳定性放大到外部世界。

并发如果不受约束，就会扩大事故半径。Claude Code 在这里采取了偏保守的策略。在会碰到文件、终端和权限的场景里，这种保守通常更可靠。

1.5 第四层 Harness：最危险的工具，必须配最细的规矩

在所有工具里，Bash 最值得警惕。因为它几乎不受领域边界约束，可以直接接触文件、进程、网络和 Git 仓库, 还会带上重定向、管道等复杂 shell 语义。一个系统如果对 Bash 过度信任，后果通常会很具体。

Claude Code 对 Bash 的态度，可以在 `src/tools/BashTool/prompt.ts:42` 往后看得很清楚。这里写了一整段操作规约，尤其是围绕 git 和 PR 的那部分：不要乱改 git config，不要跳过 hooks，不要随手 `git add .`，不要在 pre-commit 失败后用 `--amend` 把上一条提交也搭进去，不要在没有明确要求时提交，更不要默认 push。

写到这个地步，有人会觉得它过于细碎。但高风险接口通常就需要高密度约束。Bash 一旦进入真实工作流，很多规则都必须明确写出来。

Harness Engineering 的一个重要原则，就是把高风险能力包装成高约束能力。能力越强，控制越细。原因很简单：外部世界不会因为模型语气坚定，就自动原谅一次错误执行。

1.6 第五层 Harness：错误属于主路径的一部分

很多软件把失败路径看成例外，把成功路径看成正文。代理系统不能这样做。因为代理系统的失败不是偶发性的，它是一种稳定存在。模型会超 token，会触发 `prompt too long`，会撞上 `max_output_tokens`，会遇到工具拒绝、用户打断、hook 阻塞和 API 重试。要是这些东西都只在最后用几个 `catch` 打发一下，那系统表面上在运行，实际上只是不断把麻烦往后滚。

Claude Code 在 query loop 里没有这样处理。光看 `src/query.ts:453` 往后关于 autocompact 的处理，以及 `src/query.ts:592` 往后对上下文上限和阻断逻辑的注释，就能看出它把失败当作会持续发生的结构性条件来处理。

这也是 Harness 和普通助手的重要差别之一。普通助手常见的设计逻辑是先回答，错了再道歉。Harness 更强调先约束，再执行；即使出错，也要按恢复路径处理，而不是靠临场发挥补救。

一个会道歉的系统，不一定成熟。一个知道何时不该开始、何时该重试、何时该中止、何时该准确汇报失败的系统，才更接近成熟。

1.7 从源码里可以提炼出的第一个原则

到这里，第一章其实只想说一件事：

> 代理系统的关键能力是约束执行。

Claude Code 的源码在几个关键位置都指向同一个结论：

- `constants/prompts.ts` 说明 prompt 是控制平面的一部分，而不是人格装饰
- `utils/systemPrompt.ts` 说明系统行为必须有清楚的分层优先级
- `query.ts` 说明代理运行依赖持续的循环状态，而不是单次问答
- `services/tools/toolOrchestration.ts` 说明工具调用必须服从调度纪律
- `tools/BashTool/prompt.ts` 说明高风险工具必须伴随高密度约束

把这些放在一起看，就会发现 Harness Engineering 并不神秘。它只是坚持几条常被忽视的工程常识：

- 模型会犯错
- 工具会扩大错误后果

- 上下文会膨胀
- 状态会污染下一轮
- 用户会打断你
- 失败会反复出现

既然如此，系统就不能靠“聪明”维持秩序，只能靠结构维持秩序。结构不像聪明那样显眼，但通常更可靠。

下一章要谈的，是这套结构里最容易被误解的一层：system prompt。很多人把它看成人设文本，本书会说明它更接近操作系统里的规章制度。人设可以改善观感，规章才能约束机器。

第 2 章 Prompt 不是人格，Prompt 是控制平面

2.1 把 prompt 当成人设，是一种常见误会

很多人一说起 system prompt，首先想到的是一段熟悉的话术：你是谁，你擅长什么，你应该温柔、专业、简洁，最好再有一点稳定的人格。对于只负责聊天的系统，这种理解问题不大；但对一个要读文件、调工具、动 shell、处理权限、跨轮执行的代理系统来说，这种理解明显不够。

原因很简单。人设描述解决的是“它像什么”，控制平面解决的是“它能做什么、什么时候做、做错了怎么办、谁来兜底”。两者不在同一层。一个系统可以有讨人喜欢的人设，同时在执行层面缺少规矩。那种系统出事时往往会显得很真诚，因为它很会道歉。但道歉并不能替代运行时设计。

Claude Code 的实现恰好说明了这一点。它的 system prompt 是一组分层拼装的行为区块。换句话说，这里的 prompt 更接近一套运行时协议，而不是一篇人物小传。

2.2 从源码看，Claude Code 的 prompt 从一开始就是分层的

在 `src/constants/prompts.ts:444` 的 `getSystemPrompt()` 里，Claude Code 返回的是一个由多个 section 组成的数组，而不是一段完整字符串。这个细节很重要。因为一旦 prompt 变成多个块，系统就正式承认它内部包含一组职责不同的约束。

这些 section 至少包含几类东西。

首先是身份和总任务说明。在 `src/constants/prompts.ts:175`，系统说明自己是一个交互式代理，要用可用工具帮助用户完成软件工程任务。这里还顺手塞入了一些安全约束，比如不要乱猜 URL。

然后是系统级规则。在 `src/constants/prompts.ts:186` 开始，系统明确规定：

- 用户能看见的是哪些文本
- 工具调用可能触发权限审批
- 用户拒绝后不能机械重试
- tool result 和 user message 里可能混入 system-reminder
- 上下文会被自动压缩

这些内容有一个显著特征：它们并不关心模型“像不像一个聪明助手”，而是关心它是否是一个守规矩的执行体。这就是控制平面的语气，它的核心任务是定义边界。

再往下，在 `src/constants/prompts.ts:199` 开始，是做任务时的工程性指令：不要随意增加需求，不要越权优化，不要为了看起来体面而隐瞒验证失败，不要在没有必要时制造抽象。这些内容看起来像写作风格要求，其实它们和工程约束绑得很紧。一个会自动“顺手优化一切”的模型，从产品角度看也许很热情，从工程角度看则相当危险。

所以，从源码结构上就能看出来：Claude Code 的 prompt 要解决的是如何让模型在复杂运行时里遵守边界。

2.3 Prompt 的真正价值，不在文字本身，而在优先级

如果 prompt 只是存在，还不够说明问题。真正决定它是否属于控制平面的，是系统是否给它定义了严格优先级。

这一点可以看 `src/utils/systemPrompt.ts:28` 开始的 `buildEffectiveSystemPrompt()`。这段代码把 prompt 的来源明确排成一条链：

1. override system prompt
2. coordinator system prompt
3. agent system prompt
4. custom system prompt
5. default system prompt

最后还会统一拼接 `appendSystemPrompt`。

这个设计很说明问题。它表明 Claude Code 并不相信“默认 prompt 一劳永逸”。相反，它承认系统里存在多种语境：

- 协调者模式需要自己的系统行为

- agent 模式需要自己的职责说明
- 用户可以通过 CLI 覆盖或追加 prompt
- 默认 prompt 只是没有更高优先级时的基线

更朴素地说，成熟系统不会迷信唯一版本的 prompt。它会把 prompt 看成一个有层级的配置系统，让不同职责在不同上下文里生效。

这里还有一个很值得注意的细节。在 `src/utils/systemPrompt.ts:99` 往后，系统对 proactive mode 做了特殊处理：如果 agent prompt 和 proactive mode 同时存在，agent prompt 不再替换默认 prompt，而是附加在默认 prompt 之后。这个决定本身就很说明问题。它意味着系统知道，有时候默认约束不能丢，新增 agent 只能在默认约束之上叠加领域行为，而不能把整套纪律换掉。

可以把它理解为一套通用制度外加岗位说明书。岗位说明书可以补充职责，但不能直接冲掉底层制度，否则系统很快就会各自为政。

2.4 Prompt 不是静态文案，它还连接着记忆系统

如果说前面这些内容已经像一套运行时说明书，那么看到 Claude Code 如何处理 memory 和 `CLAUDE.md` 后，就会更清楚地意识到：这里的 prompt 已经是整个上下文治理入口，而不只是“写给模型看的一段话”。

在 `src/utils/claudemd.ts:1153` 的 `getClaudeMds()` 里，系统会把 project instructions、local instructions、team memory、auto memory 等不同来源的内容整理成统一格式，再拼接进 prompt 相关上下文中。这里连每种内容的来源说明都写得很细，比如这是项目级指令、用户私有项目指令、共享 team memory，还是跨会话持久化的 auto memory。

而在 `src/memdir/memdir.ts:187` 开始的 `buildMemoryLines()` 里，系统连“如何保存记忆”这件事都变成了 prompt 的一部分。它会明确告诉模型：

- memory 是文件化持久系统
- `MEMORY.md` 是索引，不是正文
- 要如何写 frontmatter
- 哪些信息不该保存
- plan 和 task 不该被误用成 memory

这件事非常关键。它把 prompt 的职责从“约束当前行为”扩展到了“约束未来知识的沉淀方式”。这已经超出了通常意义上的提示词，更接近一份写给运行时参与者的知识治理协议。

换句话说，Claude Code 不只是用 prompt 规定“这一轮怎么说话”，还用 prompt 规定“长期记忆如何形成”。一个系统只要走到这一步，它的 prompt 就不可能再只是语气问题，而必然进入制度问题。

2.5 真正的控制平面，还要考虑缓存与计算成本

多数人理解 prompt 时，很少会想到性能。常见想法是 prompt 只是喂给模型的文本，写好即可。Claude Code 的实现更直接：prompt 同时也是计算成本。它越复杂、变化越频繁，缓存命中就越差，系统运行就越贵、越慢。

在 `src/constants/systemPromptSections.ts:16` 往后，系统把 prompt section 区分成两类：

- 可缓存的 `systemPromptSection`
- 会打破缓存的 `DANGEROUS_uncachedSystemPromptSection`

而 `resolveSystemPromptSections()` 会优先从缓存里拿已经计算过的内容，只在必要时重算。到了 `clearSystemPromptSections()`，系统又会在 `/clear` 或 `/compact` 之后清空这些状态。

这件事看起来像优化，实际上同样属于控制平面。一个真正可运行的 prompt 系统，不可能只考虑表达能力，而不考虑它对吞吐、延迟和缓存的影响。Claude Code 在 `getSystemPrompt()` 里甚至把静态部分和动态部分用 boundary 显式分开，见 `src/constants/prompts.ts:560` 往后。这说明它在设计时已经承认：有些内容在会话中相对稳定，有些内容会逐轮变化，二者不能混在一起消耗缓存。

一个工程系统只要开始关心“哪部分 prompt 会导致缓存失效”，它就已经不再把 prompt 当作文案创作。文案追求完整表达，控制平面追求可治理、可复用、可预测的行为成本。两者关注的问题不同。

2.6 用户可以覆盖 prompt，但不能跳过这套结构

Claude Code 并没有把用户锁死在默认 prompt 上。相反，CLI 明确支持覆盖和追加。

在 `src/main.tsx:1342` 往后，系统处理 `--system-prompt`、`--system-prompt-file`、`--append-system-prompt`、`--append-system-prompt-file` 这些选项。也就是说，用户当然可以带着自己的规约来。

但这里有个关键点。系统虽然允许覆盖和追加，却仍然坚持用统一的 `buildEffectiveSystemPrompt()` 做最终装配。这说明它允许自定义，但不放弃秩序。用户可以改内容，系统仍然保留结构。

没有结构的可定制，最后往往会退化成另一种随意。今天加一段，明天减一段，后天某个 agent 又替换掉基线约束，系统行为就会越来越像临时口头通知。Claude Code 的选择是让用户修改，但修改必须发生在既定优先级和分层机制里。

2.7 为什么说 prompt 在这里更像宪法，而不是台词

如果把前面的几部分放在一起看，可以得到一个相当明确的结论：

Claude Code 的 prompt 更像宪法。

所谓台词，是给角色在场上说的；所谓宪法，是规定权力边界、责任关系和例外情况如何处理。Claude Code 的 prompt 更接近后者，因为它满足了几个结构条件：

- 它分层，而不是一块写到底
- 它有优先级，而不是谁后写谁说了算
- 它与 memory、CLAUDE.md、agent instructions、MCP instructions 一起组成完整控制平面
- 它有缓存和动态 section 机制，不是随手拼一段文本
- 它和 runtime 紧密耦合，而不是游离于系统之外的装饰物

这也是为什么“写一个好 prompt”单独拿出来时价值有限。更重要的问题是：prompt 在系统里处于什么位置，它和哪些模块配合，它是否参与权限、状态、上下文和长期记忆的治理。如果不回答这些问题，所谓好 prompt 往往只是在某个顺利场景里暂时成立。

2.8 从源码里可以提炼出的第二个原则

这一章最后可以归纳成一句话：

> Prompt 的价值，在于它是否被纳入一套清楚的控制结构。

Claude Code 的源码在几个地方共同证明了这一点：

- `constants/prompts.ts` 把 prompt 写成分段控制结构，而不是一段统一宣言

- `utils/systemPrompt.ts` 明确规定了 prompt 来源的优先级
- `utils/claudemd.ts` 把项目级和长期记忆内容纳入上下文装配
- `memdir/memdir.ts` 用 prompt 规定了长期记忆的保存规则
- `constants/systemPromptSections.ts` 则把 prompt 进一步变成可缓存、可失效、可按段重算的运行时对象

所以，一个成熟代理系统里的 prompt，不该被理解成“让模型入戏的开场白”。它更像一套运行中的制度文本。制度文本当然也可以写得清楚，但最重要的部分始终是约束力。

下一章要讨论的，是另一根更硬的骨头：query loop。因为再好的控制平面，最后都要落到执行循环里。prompt 规定边界，循环决定命运。一个系统最终会成为什么样子，往往体现在它每一轮如何继续、如何中断、如何恢复的那套状态机里。